

Project Title: Implement Client Script & UI Policy (Incident)

Project Documentation Summary

1. Introduction

The project “Implement Client Script & UI Policy (Incident)” is designed to control and validate Incident records in ServiceNow. The project uses a UI Policy and multiple Client Scripts to manage field behavior, enforce mandatory information, automatically control Incident values, and restrict unauthorized list editing.

The configuration focuses on High Impact incidents. When Impact is set to High, the UI Policy controls the Assignment group and Urgency fields, while Client Scripts provide additional validation and interaction controls. Testing activities are used to confirm that the configured behavior works correctly under different Incident conditions.

2. Idea Phase

The idea of the project is to provide a simple and reliable method for enforcing Incident form rules in ServiceNow. High Impact incidents require additional control so that important information is available before the Incident is saved.

Using UI Policies allows field behavior to change dynamically according to Incident conditions. Client Scripts provide additional validation when fields change, when a record is submitted, and when a user attempts to edit a field directly from a list.

3. Requirement Analysis

The project requires:

- A ServiceNow instance with administrative or configuration access.
- The Incident table for applying the UI Policy and Client Scripts.
- A UI Policy named “High Impact Control”.
- Impact = 1 – High as the UI Policy condition.
- Assignment group configured as mandatory through the UI Policy Action.
- Reverse if false enabled for the UI Policy.
- An Urgency UI Policy Action configured as read-only when the condition is true.
- An onChange Client Script for the Impact field to support automatic urgency handling.
- An onSubmit Client Script to prevent saving a High Impact Incident when Assigned To is empty.
- An onCellEdit Client Script to block direct State changes from the Incident list.
- Testing of mandatory enforcement, successful save, reverse condition, list editing, and form-based updates.

4. Project Design

The project is designed as a ServiceNow Incident control workflow:

Incident Form → UI Policy Condition → UI Policy Actions → Client Scripts → Validation and Testing

The UI Policy checks whether Impact is set to High. When the condition is met, the configured UI Policy Actions control field behavior. Client Scripts provide additional logic for changes to Impact, record submission, and list editing.

The UI Policy uses the condition Impact is 1 – High and has Reverse if false enabled. The Assignment group is configured as mandatory, while the Urgency field is controlled through a UI Policy Action that makes it read-only when the condition is met.

5. Planning and Scheduling

The project is divided into the following milestones and activities:

- Milestone 1 – Create the High Impact Control UI Policy.
- Milestone 2 / Activity 1 – Create the UI Policy Action for Urgency.
- Milestone 3 / Activity 1 – Create the onChange Client Script for Impact.
- Milestone 4 / Activity 1 – Create the onSubmit Client Script to prevent saving when Assigned To is missing.
- Milestone 5 / Activity 1 – Create the onCellEdit Client Script to prevent State changes through list editing.
- Milestone 6 / Activity 1 – Test mandatory enforcement.
- Milestone 6 / Activity 2 – Test successful save.
- Milestone 6 / Activity 3 – Test the reverse condition.
- Milestone 6 / Activity 4 – Test list edit blocking.
- Milestone 6 / Activity 5 – Test form-based State update.

The activities are performed sequentially so that each configuration can be created and verified before the testing stage.

6. Implementation

6.1 Milestone 1 – Create UI Policy

Navigate to System UI → UI Policies and click New. Create the UI Policy with the following configuration:

- Name: High Impact Control
- Table: Incident
- Active: true
- Condition: Impact is 1 – High
- UI Policy Action: Assignment group – Mandatory checked
- Reverse if false: true

Review the configuration and click Submit to save the UI Policy. The policy is intended to apply additional controls when an Incident has High Impact.

6.2 Milestone 2 – UI Policy Action

Open the High Impact Control UI Policy and scroll to the UI Policy Actions related list. Click New to create a new action.

- Field name: Urgency
- Read-only: true
- Visible: Leave unchanged

Click Submit to save the UI Policy Action. When Impact is High, the Urgency field becomes read-only. When the condition becomes false, Reverse if false allows the configured behavior to be reversed.

6.3 Milestone 3 – onChange Client Script

Navigate to System UI → Client Scripts and click New. Create the Client Script with the following details:

- Name: Auto set urgency for high impact
- Table: Incident
- Type: onChange
- Field name: Impact
- Active: true

Paste the configured script into the Script editor and click Submit. The provided activity description identifies this script as the Client Script used to automatically set urgency when Impact changes to High. The actual script code was not included in the supplied activity content, so no additional code has been added here.

6.4 Milestone 4 – onSubmit Client Script

Navigate to System UI → Client Scripts and click New. Configure the Client Script as follows:

- Name: Prevent save if Assigned To missing
- Table: Incident
- Type: onSubmit
- Active: true

The script checks whether Impact is High and Assigned To is empty. If both conditions are true, it displays an error and prevents the record from being saved.

```
function onSubmit() {
    if (g_form.getValue('impact') == '1' &&
        g_form.getValue('assigned_to') == '') {

        g_form.showErrorMessage(
            'assigned_to',
            'Assigned To is mandatory for High impact incidents.'
        );
        return false;
    }
    return true;
}
```

Click Submit to save the Client Script.

6.5 Milestone 5 – onCellEdit Client Script

Navigate to System UI → Client Scripts and click New. Configure the Client Script as follows:

- Name: Prevent state change via list edit
- Table: Incident
- Type: onCellEdit
- Field name: State
- Active: true

The script prevents users from changing the State field directly from an Incident list view.

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {

    alert('State cannot be updated using list editing. Please open the Incident.');
```

```
    callback(false);
}
```

Click Submit to save the Client Script.

7. Functional and Performance Testing

Functional Testing verifies that the configured UI Policy and Client Scripts perform the required functions correctly. The following tests are included:

7.1 Activity 1 – Test Mandatory Enforcement

1. Navigate to Incident → Create New.
2. Set Impact to High.
3. Leave Assigned To empty.
4. Click Submit.
5. Verify that the Incident is not saved.
6. Verify that an error message appears indicating that Assigned To cannot be empty.
7. Confirm that other configured UI Policy and Client Script behaviors are working as expected.

7.2 Activity 2 – Test Successful Save

8. Open the Incident form.
9. Fill the Assigned To field by selecting a user.
10. Click Submit.
11. Verify that the Incident is saved successfully without errors.
12. Confirm urgency auto-setting, read-only behavior, and State restrictions are functioning as configured.

7.3 Activity 3 – Reverse Condition Test

13. Open an existing Incident where Impact is High.
14. Change Impact from High to Medium.
15. Verify that Assigned To is no longer mandatory.
16. Verify that Urgency becomes editable.
17. Click Submit and confirm that the record saves successfully.

7.4 Activity 4 – Test List Edit Blocking

18. Navigate to Incident → All.
19. Locate an Incident.
20. Double-click the State field to edit it directly in the list.
21. Verify that an alert indicates direct State changes are not allowed.
22. Verify that the State value remains unchanged.

7.5 Activity 5 – Test Form-Based Update

23. Open the Incident record used in the previous tests.
24. Change the State field on the Incident form.
25. Click Update.
26. Verify that the State change is saved successfully.
27. Confirm that Assigned To and Urgency continue to follow the configured UI Policy and Client Script rules.

8. Result

The project demonstrates the implementation of Client Scripts and a UI Policy for controlling Incident records in ServiceNow. The High Impact Control UI Policy applies additional field controls when Impact is

High. The Assignment group is made mandatory and the Urgency field is made read-only according to the configured UI Policy Action.

The onSubmit Client Script prevents a High Impact Incident from being saved when Assigned To is empty. The onCellEdit Client Script blocks direct State changes through list editing while allowing State changes through the Incident form. Testing activities verify mandatory enforcement, successful save, reverse behavior, list edit blocking, and form-based updates.

9. Advantages

- Reduces incorrect or incomplete Incident submissions.
- Enforces important information for High Impact Incidents.
- Provides dynamic field behavior through UI Policies.
- Prevents unauthorized State changes through list editing.
- Allows controlled State updates through the Incident form.
- Provides immediate validation messages to users.
- Reduces the need for manual checking of required Incident information.
- Makes Incident handling more structured and consistent.

10. Disadvantages

- Incorrect Client Script configuration can affect Incident form behavior.
- Changes to field names or Incident configuration may require script updates.
- Users may need additional guidance when list editing is restricted.
- Multiple UI Policies and Client Scripts require proper testing to avoid conflicting behavior.
- Debugging Client Scripts may require technical knowledge of ServiceNow.

11. Conclusion

The “Implement Client Script & UI Policy (Incident)” project provides an organized approach to controlling Incident records in ServiceNow. The High Impact Control UI Policy manages field behavior based on the Impact condition, while Client Scripts provide validation and restrictions for Impact changes, record submission, and list editing.

The testing activities confirm that High Impact Incidents receive the intended controls, incomplete records are prevented from being saved, reverse conditions restore field behavior, list-based State changes are blocked, and form-based State updates remain possible.

12. Future Scope

The project can be enhanced in the future by:

- Adding more UI Policies for different Incident conditions.
- Adding additional validation rules for other important Incident fields.
- Extending Client Scripts to handle more complex Incident workflows.
- Adding user-friendly messages for different validation scenarios.
- Introducing additional controls for list editing of sensitive fields.
- Integrating the configuration with broader Incident management workflows.
- Adding more comprehensive functional and regression testing.
- Extending the solution to support additional ServiceNow tables and processes.